

Deep Homomorphism Networks (DHN)

Maehara & Nguyen, NeurIPS 2024

Paper Equations

Per-pattern contribution (Eq. 3)

$$t_\mu(F, G, u) = \sum_{\substack{\varphi \in \text{Hom}(F, G) \\ \varphi(r) = u}} \prod_{v \in V(F)} \mu_v(h(\varphi(v)))$$

Same equation, instantiated for $F \in \{C_2, C_3, C_4\}$. The cyclic self-join over `Edge` is the enumeration of $\varphi \in \text{Hom}(C_k, G)$ (closed walk $n \rightarrow \dots \rightarrow n$); the joins on `H0` fetch $h(\varphi(\cdot))$; per-position templated MLPs $\mu^{(k,i)}$ apply the per-position transforms; `sum` aggregates over φ with root $\varphi(r) = n$. One rule per pattern; the only thing that grows is the join arity.

Layer combination (Eq. 4)

$$h^{(l)}(u) = \rho \left(\left\| \bigcup_F t_\mu(F, G, u) \right\| \right)$$

Concatenate the per-pattern contributions over all $F \in \{C_2, C_3, C_4\}$ and pass through a learned MLP ρ , exactly the form used by the official DHN reference implementation (models.py).

Graph readout & classifier

$$\hat{y} = \text{Classifier} \left(\sum_{u \in V(G)} h^{(L)}(u) \right)$$

Figure: DHN (Maehara & Nguyen, NeurIPS 2024) — paper equations (left) and the corresponding Sysname DSL (right), shown for the C2:4 configuration ($F \in \{C_2, C_3, C_4\}$). Each paper equation maps to a single Sysname rule. In Row 1, the cyclic self-join over `Edge` performs the homomorphism enumeration $\varphi \in \text{Hom}(C_3, G)$ inline, the joins on `H0` fetch $h(\varphi(v))$, the per-position μ_v become templated MLPs, and the body's `sum` aggregates over φ rooted at n — meaning Eq. 3 is realised by one declarative rule. Row 2 is Eq. 4 in the form used by the official reference: concatenate the per-pattern contributions and pass through the layer MLP ρ . Row 3 is graph-level readout. Listing reproduced from nbs/paper_experiments/dhn/dhn_ghl_csl_c2_4.relnn. Supplementary material for “Incorporating Deep Learning Design in Database Queries.”

Sysname DSL

```
H0(g,n; z) :- Node(g,n; z) .
```

```
C2_Agg(g,n; sum(Mu<'C2',0>(h_n) * Mu<'C2',1>(h_v))) :-  
  Edge(g,n,v), H0(g,n; h_n), H0(g,v; h_v) .
```

```
C3_Agg(g,n; sum(Mu<'C3',0>(h_n) * Mu<'C3',1>(h_v) * Mu<'C3',2>(h_w))) :-  
  Edge(g,n,v), Edge(g,v,w), Edge(g,w,n),  
  H0(g,n; h_n), H0(g,v; h_v), H0(g,w; h_w) .
```

```
C4_Agg(g,n; sum(Mu<'C4',0>(h_n) * Mu<'C4',1>(h_v) * Mu<'C4',2>(h_w) * Mu<'C4',3>(h_p))) :-  
  Edge(g,n,v), Edge(g,v,w), Edge(g,w,p), Edge(g,p,n),  
  H0(g,n; h_n), H0(g,v; h_v), H0(g,w; h_w), H0(g,p; h_p) .
```

```
H1(g,n; Rho(Concat(z_2, z_3, z_4))) :-  
  C2_Agg(g,n; z_2), C3_Agg(g,n; z_3), C4_Agg(g,n; z_4) .
```

```
GraphEmb(g; sum(z)) :- H1(g,n; z) .
```

```
GraphLogits(g; Classifier(z)) :- GraphEmb(g; z) .
```